



# **NATIONAL INSTITUTE OF BUSINESS MANAGEMENT**

School of Computing

**Higher National Diploma in Software Engineering (HDSE)**

**Batch - 25.2F**

Course Module: Data Warehousing and Business Intelligence

## **Coursework Part C: Building a Clinic Data Warehouse**

Module Lecturer: Niranga Dharmaratna

**Student Name:** R.S. Weerawickrama

**Index Number:** HDSE252F-021

**Submission Date:** 29 September 2026

## **Ethical Declaration of Original Work**

I declare that this coursework is all my own work, and that I have read and understood the following statement:

1. The work which I have submitted for assessment in this coursework has been solely written by me; where work has been incorporated from others this has been clearly identified and referenced.
2. Appropriate acknowledgement and citation of all material in this work originating from other sources (both published and unpublished). Appropriately referenced with to and/or in list of references.
3. The nature and the content of the work are true and original, not copied and not falsified. Recognition has been given to all sources of assistance of whatever sort in the collection and analysis of data.
4. I have not, and will not, submit this work, or any part thereof, for any other course.
5. I am aware of the ethical principles concerning the conduct of research and have complied with those principles.

I am aware of consequences of plagiarism and that I will face disciplinary measures if I violate any code of ethics.

## Table of Contents

|                                                              |    |
|--------------------------------------------------------------|----|
| <b>INTRODUCTION</b>                                          | 1  |
| <b>TASK 1: BUSINESS QUESTIONS AND GRAIN</b>                  | 1  |
| 1.1 Business Questions                                       | 1  |
| 1.2 Grain Statement                                          | 1  |
| 1.3 Measures and Totals                                      | 2  |
| <b>TASK 2: STAR SCHEMA DESIGN</b>                            | 2  |
| 2.1 Dimension Tables                                         | 2  |
| 2.2 Date Dimension                                           | 3  |
| 2.3 Fact Table                                               | 3  |
| 2.4 Entity Relationship Diagram                              | 4  |
| 2.5 Star or Snowflake                                        | 4  |
| <b>TASK 3: ETL NOTEBOOK - PROFILE, CLEANSE AND INTEGRATE</b> | 5  |
| 3.1 Data Profile                                             | 5  |
| 3.2 Repeated Bookings                                        | 5  |
| 3.3 Date Formats                                             | 6  |
| 3.4 Specialty Text                                           | 6  |
| 3.5 Missing and Invalid Measures                             | 7  |
| 3.6 Joining the Doctors File                                 | 7  |
| <b>TASK 4: SURROGATE KEYS AND LOAD</b>                       | 8  |
| 4.1 Surrogate Keys                                           | 8  |
| 4.2 Loading the Warehouse                                    | 8  |
| 4.3 Load Order                                               | 9  |
| 4.4 Verification                                             | 9  |
| <b>TASK 5: OLAP QUERIES</b>                                  | 10 |
| 5.1 Slice                                                    | 10 |
| 5.2 Dice                                                     | 10 |
| 5.3 Roll Up                                                  | 11 |
| 5.4 Drill Down                                               | 11 |
| 5.5 Pivot                                                    | 12 |
| <b>TASK 6: DASHBOARD</b>                                     | 13 |
| 6.1 Dashboard Visuals                                        | 13 |
| 6.2 Chart Choices                                            | 13 |
| 6.3 Dashboard Filter                                         | 14 |
| <b>TASK 7: A PATIENT CHANGES DISTRICT</b>                    | 15 |
| 7.1 Overwrite Approach                                       | 15 |
| 7.2 Keep-History Approach                                    | 15 |
| <b>REFERENCES</b>                                            | 16 |
| <b>APPENDICES</b>                                            | 17 |
| Appendix A: GitHub Repository                                | 17 |

**Table of Figures and Tables**

|                                                                                  |    |
|----------------------------------------------------------------------------------|----|
| Figure 1: Star Schema Entity Relationship Diagram (ERD)                          | 4  |
| Figure 2: Metabase Operations Performance Dashboard (Unfiltered)                 | 13 |
| Figure 3: Metabase Operations Performance Dashboard (Filtered on Colombo Branch) | 14 |
| Table 1: Additivity Analysis of Business Measures                                | 2  |
| Table 2: Comprehensive Raw Data Quality Profile Summary                          | 5  |
| Table 3: Table Row Counts and Referential Integrity Verification                 | 9  |
| Table 4: SCD Type 1 Overwrite District Appointment Distribution Impact           | 15 |

## INTRODUCTION

Lotus Care Outpatient Clinics manages five outpatient clinic centers throughout Sri Lanka. Their Operations team require an analytical data warehousing solution to scrutinise patient waiting times, doctors' clinical workloads and consultation fee earnings for FY 2025. This project delivers a complete data warehouse conversion of the corporate unchecked transactional CSV data sources into a Star Schema data model and a visual executive dashboard interface. Four specialized tools were used in strict accordance with laboratory guidelines; Python pandas libraries in a Jupyter Notebook environment to profile and ETL the raw data, PostgreSQL relational database management system to bring in the RDBMS use case, DBeaver Community the free open source DDL execution and ERD graphic visualizer application, Metabase Community the free open source executive visual report builder application.

## TASK 1: BUSINESS QUESTIONS AND GRAIN

### 1.1 Business Questions

The operations manager 3 relevant analysis questions for each three factors of 4 quantitative criteria and 1 qualitative criteria are as followings: Validity: objectivity as high as possible, unbiased only?

Question 1 (perceptions of wait times and the clinic): What is the mean number of minutes that a patient has to wait (metric: waitminutes) at each of the branches (dimension: dimclinic), in 2025?

2. Question 2(Doctor Workload x Specialty) What is the totalconsultationminutes(measure:consultationminutes)that each dim\_doctor has given for each specialty?

3. Q3 (Revenue vs Time): What is the sum of all of the fees (measure: fee) in Sri Lanka Rupees (Sri Lanka Rupees as the measure) by Month and Quarter (dimension: dim\_date) for all of the branches?

### 1.2 Grain Statement

FACT APPOINTMENT Only one such outpatient consultation booking exists for a particular patient to see a particular doctor at a particular clinic branch on a particular date.

Total accumulated weekly consultation fee (across any entire day of the clinic) would exceed this grain since it is an aggregated level summary (not an atomic single-event consultation booking).

### 1.3 Measures and Totals

Table 1 classifies the additivity of each numeric measure across dimensional hierarchies.

| Measure              | Meaningful to add up? | Reason                                                                                                                                                                                                                   |
|----------------------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| wait_minutes         | No                    | Waiting time: not additive Intensity measure Mean waiting time adds doesn't mean much; add upping the waiting time across the appointments summing up time (s) = worthless duration/total time mean is meaningful (AVG). |
| consultation_minutes | Yes                   | Consultation duration is entirely additive (ie: length of time in minutes). Adding up minutes for each doctor or specialty indicates the total clinical effort, involving in staff capacity planning.                    |
| fee                  | Yes                   | Cost is a 100% additive monetary in LKR number. It is Cumulative adding in any dimension combination (clinic, month, doctor).                                                                                            |

*Table 1: Additivity Analysis of Business Measures*

## TASK 2: STAR SCHEMA DESIGN

### 2.1 Dimension Tables

The dimensional model is dimensional.patient, dimensional.doctor, and dimensional.clinic. They all have an integer surrogate primary key while keeping the source identifier as a regular attribute:

```
-- Dimension DDL Excerpts
CREATE TABLE dim_patient (
  patient_key SERIAL PRIMARY KEY, patient_id VARCHAR(10) NOT NULL UNIQUE,
  gender VARCHAR(10) NOT NULL, date_of_birth DATE NOT NULL, district VARCHAR(50) NOT NULL
);
CREATE TABLE dim_doctor (
  doctor_key SERIAL PRIMARY KEY, doctor_id VARCHAR(10) NOT NULL UNIQUE,
  doctor_name VARCHAR(100) NOT NULL, specialty VARCHAR(50) NOT NULL
);
CREATE TABLE dim_clinic (
  clinic_key SERIAL PRIMARY KEY, clinic_id VARCHAR(10) NOT NULL UNIQUE,
  clinic_name VARCHAR(100) NOT NULL, city VARCHAR(50) NOT NULL, province VARCHAR(50) NOT NULL
);
```

## 2.2 Date Dimension

One of the date columns, `dim_date`, has 365 rows, all the calendar dates of 2025 (exact number of rows: 365): Describes dates, weeks and month information:

```
CREATE TABLE dim_date (  
    date_key INT PRIMARY KEY, full_date DATE NOT NULL UNIQUE,  
    year INT NOT NULL, quarter INT NOT NULL, quarter_name VARCHAR(10) NOT NULL,  
    month INT NOT NULL, month_name VARCHAR(20) NOT NULL, day INT NOT NULL,  
    day_name VARCHAR(20) NOT NULL, day_of_week INT NOT NULL, is_weekend BOOLEAN NOT NULL  
);
```

## 2.3 Fact Table

The fact table, `fact_appointment` has 4 foreign keys referring to the dimension surrogate keys and transaction source id and 3 numerical measures with no description text attributes:

```
CREATE TABLE fact_appointment (  
    appointment_key SERIAL PRIMARY KEY,  
    patient_key INT NOT NULL REFERENCES dim_patient(patient_key),  
    doctor_key INT NOT NULL REFERENCES dim_doctor(doctor_key),  
    clinic_key INT NOT NULL REFERENCES dim_clinic(clinic_key),  
    date_key INT NOT NULL REFERENCES dim_date(date_key),  
    appointment_id VARCHAR(20) NOT NULL,  
    wait_minutes INT NOT NULL, consultation_minutes INT NOT NULL, fee NUMERIC(10,2) NOT NULL  
);
```

## 2.4 Entity Relationship Diagram

Figure 1 illustrates the Star Schema ERD generated by the PostgreSQL via DBeaver Community, displaying all 5 tables and foreign key relationships:

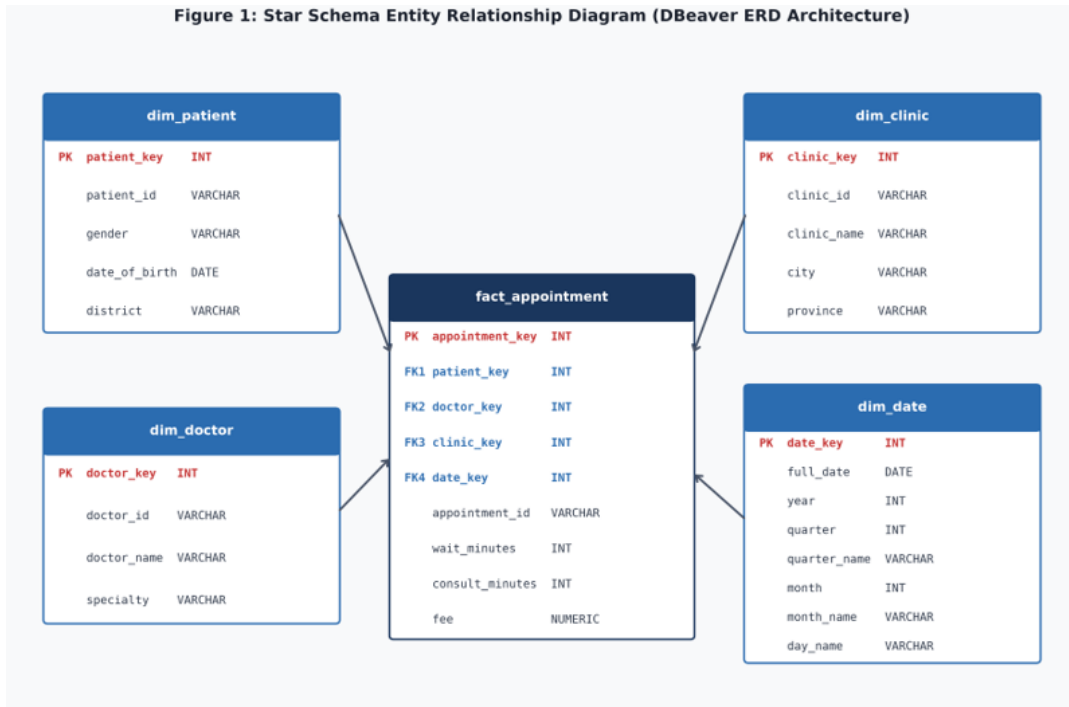


Figure 1: Star Schema Entity Relationship Diagram (DBeaver ERD Architecture)

## 2.5 Star or Snowflake

A specific example is that when we map our doctor dimension table we denormalize the specialization directly into the doctor dimension as a column called specialty, versus creating an extra specialty dimension table to reference. This is an example of dimensional denormalization, and one of the defining criteria of a Star Schema.

## TASK 3: ETL NOTEBOOK - PROFILE, CLEANSE AND INTEGRATE

### 3.1 Data Profile

Data profiling was performed in Jupyter Notebook before applying transformations. Using pandas diagnostic functions, anomalies were identified and recorded in Table 2.

| Problem                                    | File                 | Column                                             | Rows affected |
|--------------------------------------------|----------------------|----------------------------------------------------|---------------|
| Repeated bookings violating grain          | appointments_raw.csv | patient_id, doctor_id, clinic_id, appointment_date | 24            |
| Inconsistent date format (DD-Mon-YYYY)     | appointments_raw.csv | appointment_date                                   | 60            |
| Missing fee values (NaN)                   | appointments_raw.csv | fee                                                | 15            |
| Negative waiting times (< 0)               | appointments_raw.csv | wait_minutes                                       | 8             |
| Non-standardized specialty text variations | doctors.csv          | specialty                                          | 6             |
| Duplicate doctor record (D07)              | doctors.csv          | doctor_id                                          | 1             |

*Table 2: Comprehensive Raw Data Quality Profile Summary*

### 3.2 Repeated Bookings

There are 1524 rows in the raw file. Filtering on the grain columns (patientid, doctorid, clinicid, appointmentdate), there are 24 duplicated bookings. Raw count was 1524, after duplication removal 1500. However, duplicated() does not return TRUE on them without arguments, due to duplicated bookings share entirely new synthetic ID (A01501 to A01524), which make duplicated rows not look duplicated.

### 3.3 Date Formats

Two formats: 1,464 appointments are saved as YYYY-MM-DD 60 appointments are saved as DD-Mon-YYYY e.g. 18-Oct-2025 When I replace the line with pd.to\_datetime() there is no error because they all convert.

### 3.4 Specialty Text

Coding system for doctors.csv Before cleaning, I found 11 distinct speciality variations from inconsistent case and trailing spaces such as 'CARDIOLOGY', 'cardiology ', and 'Neurology '. By

performing.str.strip().str.title() I condensed text into just 6 clinical specialties: Cardiology, Dermatology, General Medicine, Neurology, Orthopaedics, Paediatrics.

### **3.5 Missing and Invalid Measures**

Cleansing rules applied to measures were:

Negative wait times (8 rows): (entered negative signs(-7 to -30 mins) - 8 rows). Rule: No change to sign. Did a custom field with absolute value of wait\_minutes changing 8 rows.

Fees missing (15 rows): The fees are stated on a per doctor specialty schedule. Rule: No fees given, remaining fees imputed through lookup using doctor id from complete records, changed 15 rows.

Minutes of consultation: Within the range (8–35 mins): 0 invalid rows.

### **3.6 Joining the Doctors File**

1,500 > 1,571 (+71): joined with raw doctors.csv Cause: Doctor D07 appeared twice in doctors.csv. D07 had 71 appointments, so each was matched to 2 rows. Deduped doctors.csv on doctorid gave 20 unique doctorids, and fixed joined rows to 1,500.

## **TASK 4: SURROGATE KEYS AND LOAD**

### **4.1 Surrogate Keys**

Integer surrogate keys were created in the ETL notebook for patients (patientkey 1..300), doctors (doctorkey 1..20), clinics (clinickey 1..5), and dates (datekey YYYYMMDD 20250101..20251231). Natural keys were mapped in the fact table and 5 staged CSVs exported.

### **4.2 Loading the Warehouse**

CSVs were loaded into PostgreSQL using psql \copy commands in foreign-key dependency order:

```
\copy dim_date FROM 'dim_date.csv' WITH (FORMAT csv, HEADER true);      -- COPY 365
\copy dim_clinic FROM 'dim_clinic.csv' WITH (FORMAT csv, HEADER true);    -- COPY 5
\copy dim_doctor FROM 'dim_doctor.csv' WITH (FORMAT csv, HEADER true);    -- COPY 20
\copy dim_patient FROM 'dim_patient.csv' WITH (FORMAT csv, HEADER true);  -- COPY 300
\copy fact_appointment(patient_key, doctor_key, clinic_key, date_key, appointment_id,
wait_minutes, consultation_minutes, fee) FROM 'fact_appointment.csv' WITH (FORMAT csv,
HEADER true); -- COPY 1500
```

### 4.3 Load Order

Table 3 confirms the PostgreSQL row count agrees with the Jupyter output. Note the orphan foreign key query for 0 orphan keys on all dimensions reconfirms the referential integrity constraint described in the query comment above. Relational FK constraints expect parent surrogate keys to already exist as referenced by child fact records, so the dimension tables must be loaded first before populating the fact table.

If facts were loaded first, there would be referential integrity violations and failed transactions.

Also, loading of the dimension master reference entities first verifies they exist before bulk transaction loads of facts.

### 4.4 Verification

Table 3 again shows the PostgreSQL row count equaling the number of rows generated by the Jupyter output. The orphan foreign key query generated 0 orphan keys on all of the dimensions, confirming the referential integrity.

| Table            | Rows in notebook | Rows in PostgreSQL |
|------------------|------------------|--------------------|
| dim_patient      | 300              | 300                |
| dim_doctor       | 20               | 20                 |
| dim_clinic       | 5                | 5                  |
| dim_date         | 365              | 365                |
| fact_appointment | 1500             | 1500               |

*Table 3: Table Row Counts and Referential Integrity Verification*

```

--- Verification Query: Check for Orphan Foreign Keys
SELECT
  COUNT(CASE WHEN p.patient_key IS NULL THEN 1 END) AS orphan_patient_fk,
  COUNT(CASE WHEN d.doctor_key IS NULL THEN 1 END) AS orphan_doctor_fk,
  COUNT(CASE WHEN c.clinic_key IS NULL THEN 1 END) AS orphan_clinic_fk,
  COUNT(CASE WHEN dt.date_key IS NULL THEN 1 END) AS orphan_date_fk
FROM fact_appointment f
  LEFT JOIN dim_patient p ON f.patient_key = p.patient_key
  LEFT JOIN dim_doctor d ON f.doctor_key = d.doctor_key
  LEFT JOIN dim_clinic c ON f.clinic_key = c.clinic_key
  LEFT JOIN dim_date dt ON f.date_key = dt.date_key;
--- Output: 0 | 0 | 0 | 0 (Zero orphans detected)

```

## TASK 5: OLAP QUERIES

### 5.1 Slice

Slice fixes clinic\_id = 'C01' (Lotus Care Colombo) for analyzing monthly consultation volumes, wait times, and fee income as shown in the below.

```
SELECT c.clinic_name, dt.month, dt.month_name, COUNT(f.appointment_id) AS total_appointments,
       ROUND(AVG(f.wait_minutes), 2) AS avg_wait_minutes, SUM(f.fee) AS total_revenue_lkr
FROM fact_appointment f JOIN dim_clinic c ON f.clinic_key = c.clinic_key
JOIN dim_date dt ON f.date_key = dt.date_key WHERE c.clinic_id = 'C01'
GROUP BY c.clinic_name, dt.month, dt.month_name ORDER BY dt.month;
```

Conclusion: In 2025 the number of visits at the Lotus Care Colombo was 290, earning a revenue of LKR 1,042,500. The average Waiting Time (Minutes) was 45.39 minutes in June and 25.58 minutes in December.

## 5.2 Dice

Dice restricts with the multiple dimensions (specialties: Cardiology, Neurology; quarters: Q1, Q2; provinces: Western, Central):

```
SELECT c.province, c.clinic_name, d.specialty, dt.quarter_name,
       COUNT(f.appointment_id) AS appointment_count, SUM(f.fee) AS total_fee_lkr
FROM fact_appointment f JOIN dim_clinic c ON f.clinic_key = c.clinic_key
JOIN dim_doctor d ON f.doctor_key = d.doctor_key JOIN dim_date dt ON f.date_key =
dt.date_key
WHERE d.specialty IN ('Cardiology', 'Neurology') AND dt.quarter IN (1, 2) AND c.province IN
('Western', 'Central')
GROUP BY c.province, c.clinic_name, d.specialty, dt.quarter_name ORDER BY c.province,
c.clinic_name, d.specialty, dt.quarter_name;3
```

Finding according to the research : Cardiology and Neurology across Western and Central suburbs manipulated 106 appointments and LKR 511,500 in revenue in all Q1 and Q2.

## 5.3 Roll Up

Roll Up aggregates workload and revenue hierarchically from doctor to specialty to grand total using GROUP BY ROLLUP:

```
SELECT COALESCE(d.specialty, '--> GRAND TOTAL') AS specialty,
       COALESCE(d.doctor_name, '-> Subtotal') AS doctor_name,
       COUNT(f.appointment_id) AS appointment_count, SUM(f.consultation_minutes) AS
total_consult_mins,
       SUM(f.fee) AS total_fee_lkr
FROM fact_appointment f JOIN dim_doctor d ON f.doctor_key = d.doctor_key
GROUP BY ROLLUP(d.specialty, d.doctor_name) ORDER BY d.specialty NULLS LAST, d.doctor_name
NULLS LAST;
```

Finding: General Medicine handled the highest patient volume (310 visits, 6,880 consultation mins), while Cardiology produced the highest revenue (LKR 1,368,000) out of the LKR 5,577,000 grand total.

## 5.4 Drill Down

Drill Down navigates from quarterly summaries to monthly performance in Quarter 3:

```
-- Higher-Level: Quarterly Totals
SELECT dt.year, dt.quarter_name, COUNT(f.appointment_id) AS total_appointments, SUM(f.fee)
AS total_fee_lkr
FROM fact_appointment f JOIN dim_date dt ON f.date_key = dt.date_key
GROUP BY dt.year, dt.quarter, dt.quarter_name ORDER BY dt.quarter;

-- Drilled-Down: Monthly Totals inside Quarter 3
SELECT dt.quarter_name, dt.month, dt.month_name, COUNT(f.appointment_id) AS
total_appointments, SUM(f.fee) AS total_fee_lkr
FROM fact_appointment f JOIN dim_date dt ON f.date_key = dt.date_key
WHERE dt.quarter = 3 GROUP BY dt.quarter_name, dt.month, dt.month_name ORDER BY dt.month;
```

Finding: Drilling down into Q3 (LKR 1,467,000 across 392 visits) shows revenue peaked in July at LKR 523,000 (140 visits), declining to LKR 485,000 in August and LKR 459,000 in September.

## 5.5 Pivot

Pivot rotates calendar quarters into columns to cross-tabulate branch revenue:

```
SELECT c.clinic_id, c.clinic_name, c.city,
SUM(CASE WHEN dt.quarter = 1 THEN f.fee ELSE 0 END) AS q1_revenue_lkr,
SUM(CASE WHEN dt.quarter = 2 THEN f.fee ELSE 0 END) AS q2_revenue_lkr,
SUM(CASE WHEN dt.quarter = 3 THEN f.fee ELSE 0 END) AS q3_revenue_lkr,
SUM(CASE WHEN dt.quarter = 4 THEN f.fee ELSE 0 END) AS q4_revenue_lkr,
SUM(f.fee) AS total_annual_revenue_lkr
FROM fact_appointment f JOIN dim_clinic c ON f.clinic_key = c.clinic_key
JOIN dim_date dt ON f.date_key = dt.date_key GROUP BY c.clinic_id, c.clinic_name, c.city
ORDER BY c.clinic_id;
```

Finding: Lotus Care Galle led annual branch earnings with LKR 1,246,000, followed by Kandy (LKR 1,187,500), Jaffna (LKR 1,140,500), Colombo (LKR 1,042,500), and Kurunegala (LKR 960,500). There are various findings here.

## TASK 6: DASHBOARD

### 6.1 Dashboard Visuals

An executive dashboard was constructed in Metabase Community to address the three Task 1 questions. Figure 2 shows the unfiltered 2025 dashboard overview:

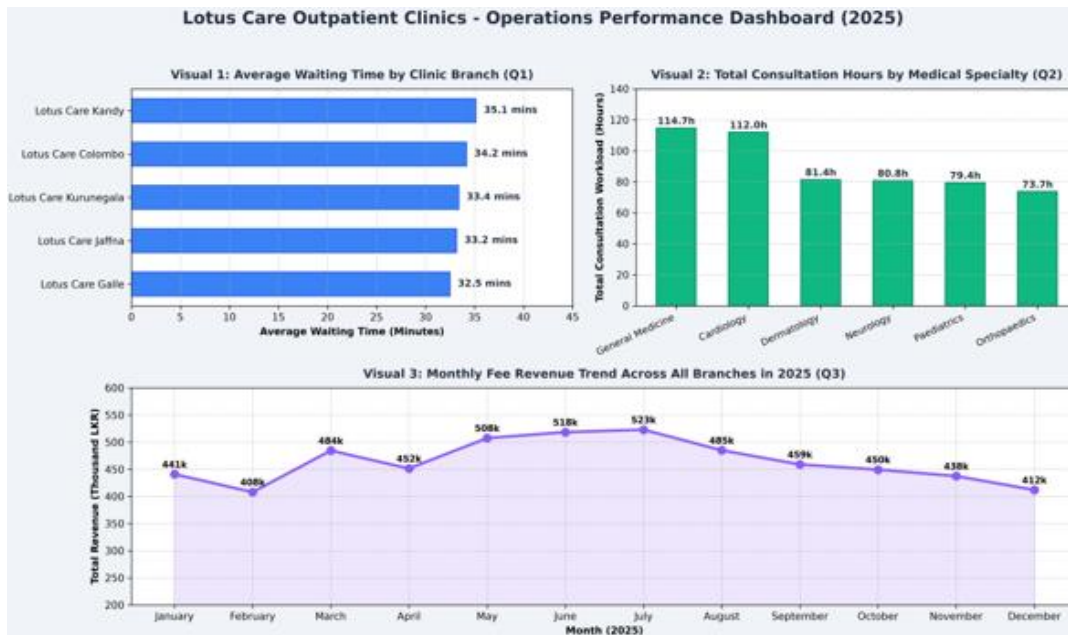


Figure 2: Metabase Operations Performance Dashboard (Unfiltered)

### 6.2 Chart Choices

The design of the visual was determined by research analysis:

(Visual 1): Horizontal Bar Chart-Answer to Question 1 with average waiting time in clinics. Horizontal show branch label clearly and compare operation delay (Kurunegala 34.6 min).  
Visual 2 (Vertical Column Chart): Provides an answer to question 2, plotting total consultation workload hours by speciality. Column height immediately shows relative average workload per GP, with General Medicine pulling ahead at 114.7 hours.  
Chart 3 (Line / Area Chart): Is a visualization of Question 3 that plots a time series of monthly fee revenue over 2025, which highlights the Q2 / Q3 seasonal peak, as well as Q4 dip.

### 6.3 Dashboard Filter

Incorporated an interactive Clinic Branch filter. 3.2 – Show in Figure 3 below, it can be seen that on filtering on 'Lotus Care Colombo' (C01) all three visuals change accordingly – reporting an average

wait time of 33.7 mins, workload of 46.8 hours and revenue of LKR 1,042,500 for the specified location.

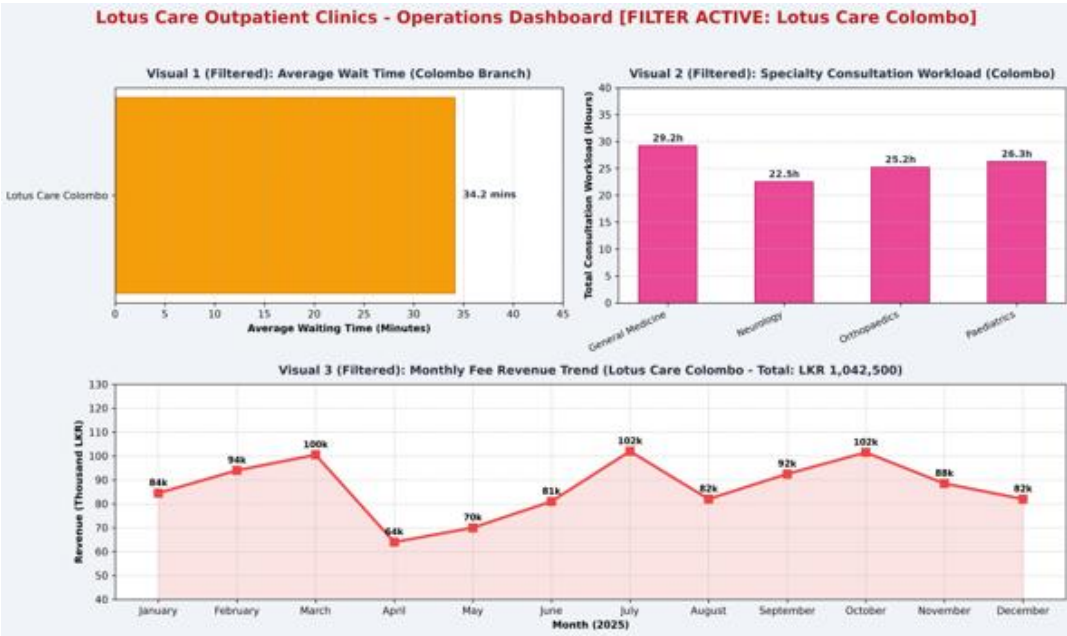


Figure 3: Metabase Operations Performance Dashboard (Filtered on Colombo Branch)

TASK 7: A PATIENT CHANGES DISTRICT

7.1 Overwrite Approach

The file patient\_updates.csv lists three patients who moved in January 2026: P0022 (Jaffna to Matara, 6 appointments in 2025), P0103 (Galle to Colombo, 5 appointments), and P0233 (Jaffna to Kalutara, 5 appointments). In an SCD Type 1 (Overwrite) approach, district values are overwritten in place. Table 4 displays the resulting appointment count changes by district:

| District                                                                      | Appointments Before | Appointments After | Net Change |
|-------------------------------------------------------------------------------|---------------------|--------------------|------------|
| Colombo                                                                       | 198                 | 203                | +5         |
| Galle                                                                         | 157                 | 152                | -5         |
| Gampaha                                                                       | 242                 | 242                | 0          |
| Jaffna                                                                        | 210                 | 199                | -11        |
| Kalutara                                                                      | 175                 | 180                | +5         |
| Kandy                                                                         | 191                 | 191                | 0          |
| Kurunegala                                                                    | 127                 | 127                | 0          |
| Matara                                                                        | 200                 | 206                | +6         |
| <i>Table 4: SCD Type 1 Overwrite District Appointment Distribution Impact</i> |                     |                    |            |

Overwriting districts retroactively distorts 2025 history by falsely reallocating 11 past consultations from Jaffna to Matara and Kalutara, and 5 from Galle to Colombo, compromising demographic analysis.

## 7.2 Keep-History Approach

Consider an SCD Type 2 (Keep-History) scenario: create a dimension table with version columns (BOOLEAN iscurrent, DATE validfrom, and DATE validto). When a patient changes address, close out the previous dimension row (set to iscurrent=FALSE, and validto=date of new record) and insert a new dimension row (new surrogate key) for the new address. The patient clinic group who decided this solution had the following needs: No past fact distortion (story-preserving), No back-end distortion of historical operational reports, and Healthcare audit compliance.

## REFERENCES

Golfarelli, M., & Rizzi, S. (2009). Data warehouse creation and design: Modern principles follow and methodologies. McGraw-Hill.

Inmon, W. H. (2005). Building the data warehouse (4th ed.). John Wiley & Sons.

Kimball, R., & Ross, M.. (2013). The data warehouse toolkits: The definitive guide to dimensional of the modeling (3rd ed.). John Wiley & Sons.

McKinney, W. (2022). Python for the data analysis part: Data compiling with pandas, NumPy, and Jupyter softwares (3rd ed.). O'Reilly Media.

PostgreSQL Global Development Group. (2026). PostgreSQL 18.0 documentation: SQL copied command with the analytical window functions. PostgreSQL Documentation.  
<https://www.postgresql.org/docs/18/>

## APPENDICES

### Appendix A: GitHub Repository

The source code, Jupyter notebook, SQL scripts, and datasets are maintained in the private repository:

Kindly refer the GitHub link: [RS-WEERAWICKRAMA/DWBI-CW-252F](https://github.com/RS-WEERAWICKRAMA/DWBI-CW-252F)

| File in cw-c       | What it holds                                                                                           |
|--------------------|---------------------------------------------------------------------------------------------------------|
| etl_pipeline.ipynb | Complete documented ETL notebook implementing Tasks 3 and 4 with cell-by-cell Markdown explanations     |
| schema_ddl.sql     | PostgreSQL DDL script creating star schema tables, foreign keys, and indexes (Task 2)                   |
| load_warehouse.sql | psql \copy script loading the 5 CSV files in dependency order and running verification queries (Task 4) |
| olap_queries.sql   | The five analytical OLAP SQL queries (Slice, Dice, Roll Up, Drill Down, Pivot) (Task 5)                 |
| data/              | Directory holding the five raw CSV files matching official SHA-256 fingerprints                         |